

Adapting OOPS concept to Backtest

In [25]: *## Import all the required libraries*

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plot
import warnings
warnings.filterwarnings("ignore")
import yfinance as yf
import quantstats as qs
import datetime
```

In [87]: `ticker = "MSFT"`
`end = datetime.date(2026,9,25)`
`start = end - pd.Timedelta(days = 365*10 )`
`sma = 50`
`lma = 100`

In [63]: `class Backtestingcrossover:`

```
    def __init__(self,ticker,start,end,sma,lma):
        self.ticker = ticker
        self.start = start
        self.end = end
        self.sma = sma
        self.lma = lma
        self.fetch_data()
        self.indicator()
        self.signal()
        self.position()
        self.returns()

    def fetch_data(self):
        self.df = yf.download(self.ticker, self.start, self.end)
        self.df = self.df.droplevel(1,axis=1)

    def indicator(self):
        self.df["sma"] = self.df["Close"].rolling(window = self.sma).mean()
        self.df["lma"] = self.df["Close"].rolling(window = self.lma).mean()

    def signal(self):
        self.df["signal"] = np.where((self.df["sma"] > self.df["lma"]) &
                                     (self.df["sma"].shift(1) < self.df["lma"])).s
        self.df["signal"] = np.where((self.df["sma"] < self.df["lma"]) &
                                     (self.df["sma"].shift(1) > self.df["lma"])).s

    def position(self):
        self.df["position"] = self.df["signal"].replace(to_replace=0 , method =

    def returns(self):
        self.df["asset_return"] = np.log(self.df["Close"]/self.df["Close"].shift
        self.df["strategy_return"] = self.df["asset_return"] * self.df["position"]
        print("total returns:", np.round(self.df["strategy_return"].cumsum()[-1]
        return self.df["strategy_return"].cumsum()[-1]
```

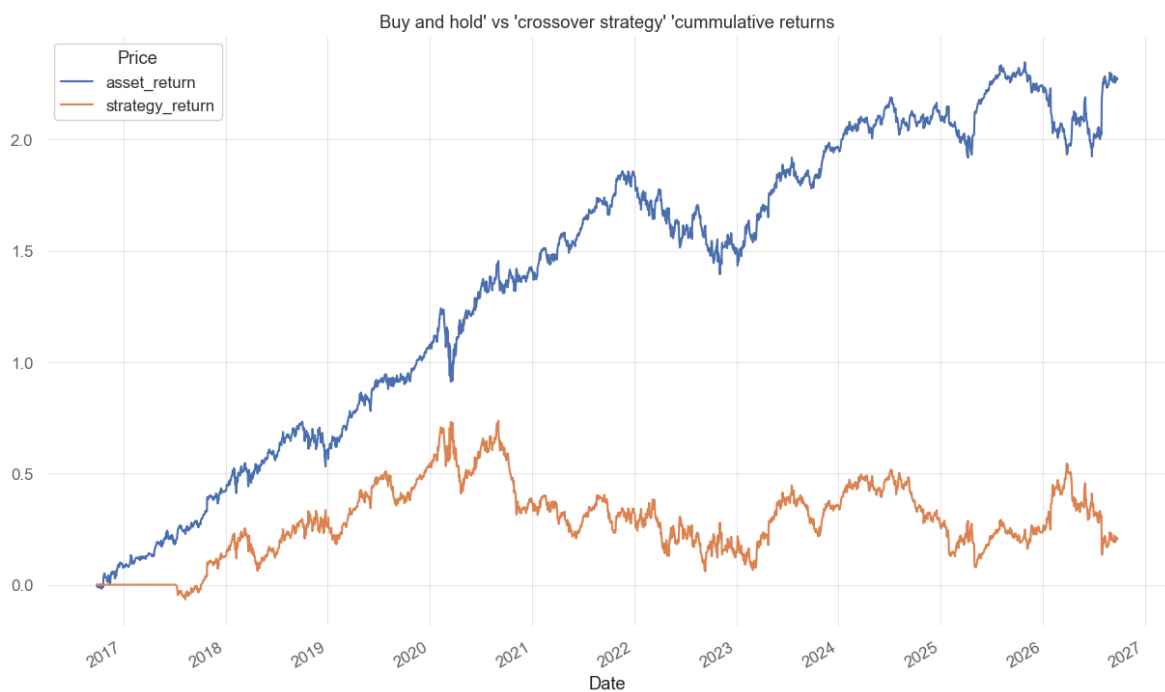
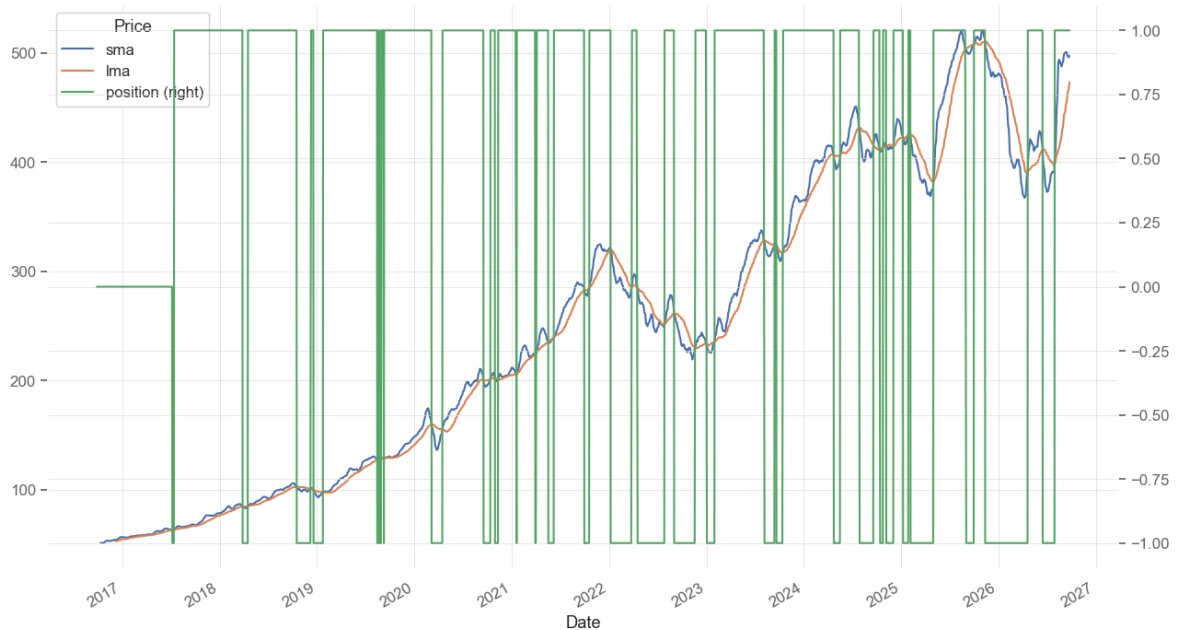
```
def analysis(self):
    self.df[["sma", "lma", "position"]].plot(figsize = (15,9) , secondary_y
    self.df[["asset_return", "strategy_return"]].cumsum().plot(figsize=(15,9)
    plot.title("Buy and hold' vs 'crossover strategy' 'cummulative returns")
    plot.show()

    qs.reports.basic(self.df["strategy_return"])
```

In [67]: data = Backtestingcrossover(ticker,start,end, sma, lma)

[*****100%*****] 1 of 1 completed
total returns: 0.21

In [68]: data.analysis()



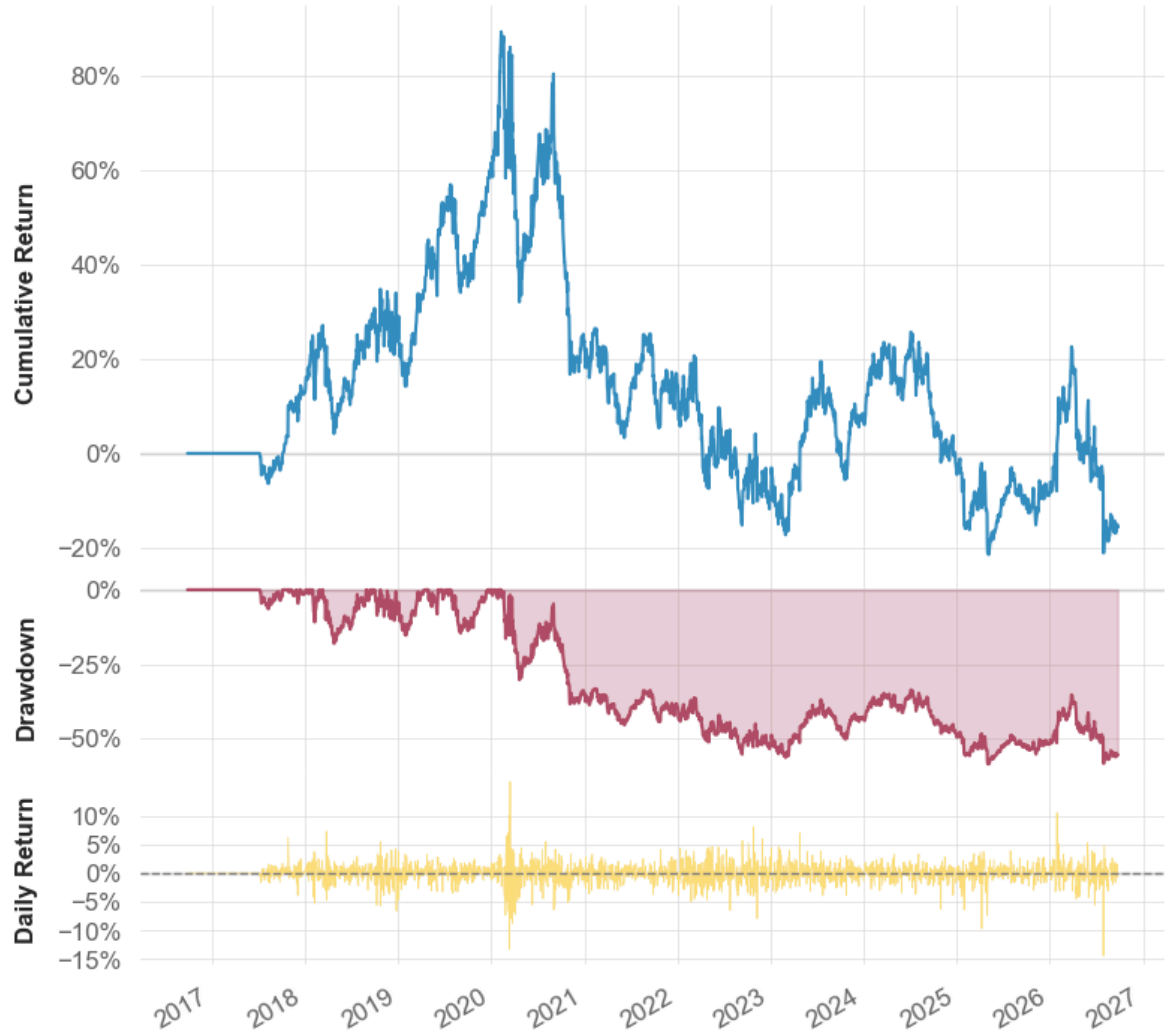
Performance Metrics

Parameter	Value
-----	-----
Risk-Free Rate	0.0%
Periods/Year	252
Compounded	Yes
Match Dates	Yes
	Strategy
-----	-----
Start Period	2016-09-28
End Period	2026-09-24
Risk-Free Rate	0.0%
Time in Market	92.0%
Cumulative Return	-15.67%
CAGR %	-1.7%
Sharpe	0.08
Prob. Sharpe Ratio	59.37%
Sortino	0.1
Sortino/√2	0.07
Omega	1.01
Max Drawdown	-58.48%
Max DD Date	2025-05-06
Max DD Period Start	2020-02-11
Max DD Period End	2026-09-24
Longest DD Days	2418
Gain/Pain Ratio	0.01
Gain/Pain (1M)	0.07
Payoff Ratio	0.97
Profit Factor	1.01
Common Sense Ratio	0.94
CPC Index	0.5
Tail Ratio	0.93
Outlier Win Ratio	4.12
Outlier Loss Ratio	3.88
MTD	-1.99%
3M	-15.47%
6M	-26.21%
YTD	-8.18%
1Y	-7.08%
3Y (ann.)	-4.08%
5Y (ann.)	-4.56%
10Y (ann.)	-1.7%
All-time (ann.)	-1.7%
Avg. Drawdown	-4.58%
Avg. Drawdown Days	83
Recovery Factor	0.35
Ulcer Index	0.35
Serenity Index	0.02

Strategy Visualization

Portfolio Summary

27 Sep '16 - 24 Sep '26 ; Sharpe: 0.08



Strategy - Monthly Returns (%)

2016	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
2017	0.00	0.00	0.00	0.00	0.00	0.00	-4.72	3.31	-0.43	11.42	1.64	1.47
2018	10.93	-1.37	-7.16	-7.81	6.06	-0.34	7.37	6.23	1.70	-0.43	-4.49	0.71
2019	-6.32	7.60	5.14	10.63	-5.15	8.05	1.63	-10.17	2.88	2.94	5.91	4.13
2020	7.75	-5.20	-2.88	-10.60	2.37	10.57	0.37	9.85	-13.83	-14.61	-7.66	3.78
2021	-3.22	0.25	-2.18	-4.34	-6.55	5.93	5.08	6.06	-6.76	-1.83	-0.25	1.42
2022	-6.07	3.40	-5.03	-8.62	1.23	5.30	-6.08	-6.91	7.31	-0.61	-0.01	-6.27
2023	-3.81	-3.39	15.21	6.12	6.92	3.45	-1.66	-1.37	-7.75	-3.80	12.11	-0.84
2024	5.60	4.07	1.53	1.20	-8.78	7.58	-2.85	-0.05	-6.85	-9.77	2.26	-4.16
2025	-11.64	3.36	5.47	-5.85	-4.17	7.99	7.12	-4.97	-2.88	-0.85	2.19	1.61
2026	11.57	8.86	5.86	-16.49	10.31	-8.30	-20.89	9.04	-1.99	0.00	0.00	0.00
	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC

In []:

Strategy optimization

```
In [75]: sma_list = [5,10,20,25]
lma_list = [50,100,200,300]

SMA = []
LMA = []
net_returns = []

for i in sma_list:
    for j in lma_list:
        print("For", i , j)
        a = Backtestingcrossover(ticker,start,end,i,j)
        SMA.append(i)
        LMA.append(j)
        net_returns.append(a.returns())

results = pd.DataFrame({"SMA" : SMA , "LMA" : LMA, "net_returns": net_returns})
results.sort_values(by="net_returns",ascending=False)
```

```
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
```

```
For 5 50
total returns: 0.01
total returns: 0.01
For 5 100
total returns: 0.62
total returns: 0.62
For 5 200
```

```
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
```

```
total returns: 0.85
total returns: 0.85
For 5 300
total returns: 0.01
total returns: 0.01
For 10 50
total returns: 0.38
total returns: 0.38
For 10 100
```

```
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
```

```
total returns: 0.39
total returns: 0.39
For 10 200
total returns: 0.74
total returns: 0.74
For 10 300
total returns: 0.18
total returns: 0.18
For 20 50
```

```
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
```

```

total returns: -0.53
total returns: -0.53
For 20 100
total returns: 0.16
total returns: 0.16
For 20 200
total returns: 1.17
total returns: 1.17
For 20 300

```

```

[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed

```

```

total returns: 0.06
total returns: 0.06
For 25 50
total returns: -0.17
total returns: -0.17
For 25 100
total returns: 0.48
total returns: 0.48
For 25 200
total returns: 1.17
total returns: 1.17
For 25 300

```

```

[*****100%*****] 1 of 1 completed

```

```

total returns: -0.18
total returns: -0.18

```

Out[75]:

	SMA	LMA	net_returns
10	20	200	1.171617
14	25	200	1.171187
2	5	200	0.852934
6	10	200	0.738797
1	5	100	0.618890
13	25	100	0.481167
5	10	100	0.386715
4	10	50	0.378289
7	10	300	0.177698
9	20	100	0.158040
11	20	300	0.062467
3	5	300	0.013647
0	5	50	0.010460
12	25	50	-0.167612
15	25	300	-0.177904
8	20	50	-0.530727

Testing the same strategy on various assets

```
In [82]: stock_list = ["BAJFINANCE.NS" , "BPCL.NS" , "CIPLA.NS" , "GAIL.NS" , "GRASIM.NS"
stock_name = []
net_returns = []
for x in stock_list:
    print("Backtesting results for " ,x)
    a = Backtestingcrossover(x, start, end , 5 , 20)
    stock_name.append(x)
    net_returns.append(a.returns())

outcome = pd.DataFrame({"stock_name" : stock_name , "net_returns" : net_returns})
outcome
```

```
[*****100%*****] 1 of 1 completed
```

```
[*****100%*****] 1 of 1 completed
```

```
Backtesting results for BAJFINANCE.NS
```

```
total returns: 1.79
```

```
total returns: 1.79
```

```
Backtesting results for BPCL.NS
```

```
[*****100%*****] 1 of 1 completed
```

```
[*****100%*****] 1 of 1 completed
```

```
[*****100%*****] 1 of 1 completed
```

```
total returns: -1.23
```

```
total returns: -1.23
```

```
Backtesting results for CIPLA.NS
```

```
total returns: -0.65
```

```
total returns: -0.65
```

```
Backtesting results for GAIL.NS
```

```
total returns: -0.85
```

```
total returns: -0.85
```

```
Backtesting results for GRASIM.NS
```

```
total returns: 0.53
```

```
total returns: 0.53
```

```
Backtesting results for COALINDIA.NS
```

```
[*****100%*****] 1 of 1 completed
```

```
total returns: 0.11
```

```
total returns: 0.11
```

```
Out[82]:
```

	stock_name	net_returns
0	BAJFINANCE.NS	1.789225
1	BPCL.NS	-1.233156
2	CIPLA.NS	-0.653620
3	GAIL.NS	-0.851503
4	GRASIM.NS	0.531808
5	COALINDIA.NS	0.106391

```
In [ ]:
```

Using Inheritance , static method and class method to create new/modify existing strategy

lets suppose we want to test same strategy but with EMA

```
In [95]: class Backtestingema(Backtestingcrossover):

    def indicator(self):
        self.df["sma"] = self.df["Close"].ewm(span = self.sma, adjust=False).mean()
        self.df["lma"] = self.df["Close"].ewm(span = self.lma, adjust=False).mean()

    @classmethod
    def about_this_strategy(cls):
        print("we are backtesting EMA cross over")
```

```
In [96]: data = Backtestingcrossover(ticker,start,end, sma, lma)
```

```
[*****100%*****] 1 of 1 completed
total returns: 0.95
```

```
In [97]: data1 = Backtestingema(ticker,start,end,sma,lma)
```

```
[*****100%*****] 1 of 1 completed
total returns: 1.76
```

```
In [ ]:
```